

Compile, Then Page: Executable SOP Programs and a Capability-Gated Runtime for Procedural LLM Agents

Chenglin Yu¹, Li Yin², Ying Yu³, Hongxia Yang⁴, Ming Li^{1,5}

¹Department of Industrial and Systems Engineering, The Hong Kong Polytechnic University

²Department of Data and Systems Engineering, The University of Hong Kong

³College of Economics and Management, Zhejiang Normal University

⁴Department of Computing, The Hong Kong Polytechnic University

⁵Research Institute for Generative AI, The Hong Kong Polytechnic University

Correspondence: ming.li@polyu.edu.hk

Abstract

Enterprise agents must follow long-horizon, conditional, safety-critical standard operating procedures (SOPs). We compile machine-readable SOP constraints into executable pseudo-code and run them with a program-guided (PG) stack machine that pages the active frame while an LLM performs semantic execution. A three-arm SOPBench study across six models separates representation from runtime: compiled text never significantly hurts and gains up to 16.0 points where official prose underperforms. Runtime guidance is capability-gated. Two strong models independently show positive seven-domain PG contrasts (58:19 and 75:31 discordant pairs), whereas weak models are harmed. A full-program cursor ablation—active frame first, complete program retained—recovers much of the strong-model refusal gain; selective visibility adds a smaller improvement. Paired probe and audit measurements track this divide to *spontaneous* state discipline rather than reconstruction ability. On Bank the three primary arms rise 70.4→86.4→92.8, with 100% refusal correctness. Practical guidance: compile first; enable active-frame paging only after a model-level discipline check.

1 Introduction

Deploying language agents in customer-facing operations is, to a first approximation, a procedure-following problem. Banks, clinics, and service desks encode their obligations as standard operating procedures (SOPs): long-horizon workflows with conditional branches and safety-critical refusal rules—what to do, what to verify first, when to decline. On SOPBench [Liu et al., 2025], the best official configuration passes 82.4% of Bank tasks on the subset we study, with the best refusal accuracy reaching 90.7%—an error rate a compliance office would not accept.

The dominant practice puts the entire SOP into the prompt as text. Resident prose ($\sim 10^4$ characters per turn on Bank) competes with the dialog for attention [Liu et al., 2024a], and verbalization strips executable structure: alternative order, gate

short-circuits, the boundary between *what must hold* and *how to verify it*. The symptoms: skipped checks, premature actions, incorrect refusals.

We treat the SOP as a program rather than a text. An offline, deterministic compiler translates the benchmark’s machine-readable dependency constraints—the same source its verbalizer renders as the baseline prose—into a two-layer pseudo-code program: process functions, one per user goal, and rule sub-routines pairing each constraint with its verification recipe and an evidence-bearing return. An online *program-guided* (PG) runtime executes it as a two-layer virtual machine: a symbolic stack machine does the bookkeeping (stack, cursor, variables, recovery) and pages *only the active frame* into context, while the LLM converses, calls tools, and judges evidence. Enforcement is deliberately *soft*—attention, not permissions—at the price that correctness depends on faithful local-frame execution: a design bet we measure.

A three-arm design (official text; compiled text; compiled program plus runtime) separates the two layers on identical tasks, and six models spanning capability tiers chart where each layer pays. The compiled representation never significantly hurts and pays where official prose underperforms; a content×format factorial shows the code-format package is likewise model-dependent. Runtime guidance divides more sharply: two strong models independently improve across seven domains, while weak models lose 14–26 points on Bank. A full-program cursor control localizes the strong-model benefit: making the active frame salient while retaining the global program recovers most of the refusal gain, and selective visibility adds a smaller increment. Paired probe-and-audit measurements trace the capability gate to spontaneous state discipline, corroborated at the attention level.

Our contributions are: (1) an SOP-to-program compiler and stack-paged runtime that make procedures executable and auditable; (2) a six-model attribution study whose seven-domain matrix independently reproduces positive PG effects for two strong models (58:19 and 75:31) and negative effects below the observed capability gate; (3) mechanism controls separating generic bookkeeping, active-frame salience, and selective visibility, alongside reconstruction, natural-audit, and attention

diagnostics; and (4) a deployment rule—compile first, then page only after a model-level discipline check—plus a benchmark defect report.

2 Related Work

Progressive disclosure and context engineering. Managing what enters an agent’s context is an established lever: resident long text is not reliably used [Liu et al., 2024a], virtual context managers page *memory* in and out [Packer et al., 2023], and Agent Skills [Anthropic, 2025] disclose capability documents progressively at the *document* level. InfiAgent externalizes persistent agent state to files [Yu et al., 2026]; we use its public runtime substrate but add policy compilation and active-frame control. Our runtime pages *policy* rather than *memory*, at the *instruction* level, with call-graph-following, stateful, revocable disclosure (Method).

Programs that control language models. DSPy [Khat-tab et al., 2024], LMQL [Beurer-Kellner et al., 2023], and SGLang [Zheng et al., 2024] compile or script *pipelines of LM calls*: the program lives outside the model and invokes it. Finite-state scaffolds are an older form: task-oriented dialog systems pair a symbolic dialog manager with a statistical language layer [Young et al., 2013]; StateFlow [Wu et al., 2024] routes an LLM through hand-designed states. We invert this relationship: the program is the artifact the model must follow, and a symbolic runtime feeds it to the model frame-by-frame while the model remains the semantic executor inside the program.

SOP-following agents and hard enforcement. SOP-driven agents encode procedures as prompts or role scripts [Ye et al., 2025, Hong et al., 2024], inheriting the resident-text failure modes quantified by our offtext arm; FlowAgent [Shi et al., 2025] adds decision controllers over a procedure language, versus our stack-paged compiled program. At the opposite extreme, constrained decoding and FSM-style controllers [Willard and Louf, 2023] enforce structure *hard*; this suits output formats, but procedures unfold over dialog turns where users change goals and evidence arrives in several valid orders, and a wrongly blocked action is unrecoverable. Our soft enforcement occupies the middle: every deviation remains possible but observable in the stack audit trail, and the cost of this flexibility—dependence on local instruction-following—is exactly the capability gate we characterize.

Agent benchmarks. General agent benchmarks measure tool competence [Liu et al., 2024b, Yao et al., 2023] or user-interaction reliability [Yao et al., 2025]; SOPBench [Li et al., 2025] grades against an oracle dependency graph with dual execute/refuse task types; we also report two confirmed simulator defects and three transparently screened omissions.

3 Method

We consider an agent that must complete user requests in a tool-augmented environment while complying with an SOP. Our approach replaces the dominant “SOP as prompt text” practice with a two-stage design: an offline *compiler* turns the SOP into an executable pseudo-code program, and an online *program-guided (PG) runtime* executes that program as a two-layer virtual machine, in which a symbolic runtime performs syntactic bookkeeping while the LLM acts as the semantic executor. The final subsection explains why enforcement is *soft*; this choice preserves flexibility, and it predicts the capability threshold that Results observes. Figure 1 gives an overview.

3.1 Compiling SOPs into Executable Programs

The official baseline verbalizes the same dependency trees used by our compiler, but drops alternative order, gate short-circuits, and the separation between what must hold and how to verify it. The compiler instead emits a two-layer program: one *process function* per user goal calls *rule subroutines* that pair the official constraint wording with a concrete verification recipe and return $\{\text{holds}, \text{evidence}\}$. A false required result branches to *fail* before the goal action. Figure 2 shows a real artifact and audit trail. Five mechanisms make the transformation executable rather than cosmetic:

1. **Stateful subroutines.** Interaction-requiring constraints compile to multi-turn functions whose progress persists across turns.
2. **Transitive verifier-gate hoisting.** Login/authentication gates required by verification actions are closed transitively and placed before guarded checks.
3. **Two disjunctions.** We adopt an evidence-complete schedule for policy *or*; this author-side convention preserves the benchmark’s Boolean truth condition but is not required by its oracle. Alternative tool recipes are semantically equivalent paths and are robustness-ranked (full-state reads first).
4. **Frame factoring.** Large compound groups and composite alternatives become named functions, keeping continuations explicit.
5. **Evidence-bearing returns.** Semantic identifiers and explicit return contracts keep every verdict tied to its verifier.

Formalization and guarantees. A leaf is a literal $\ell = (c, \theta, q)$: predicate c , parameter binding θ , and required polarity $q \in \{0, 1\}$ ($q = 0$ denotes *not* c). A rule is polarity-neutral: it returns $r = (h, e)$, where h is the observed truth of $c(\theta)$ and e is its tool evidence; ℓ holds iff $h = q$. For any evidence-faithful

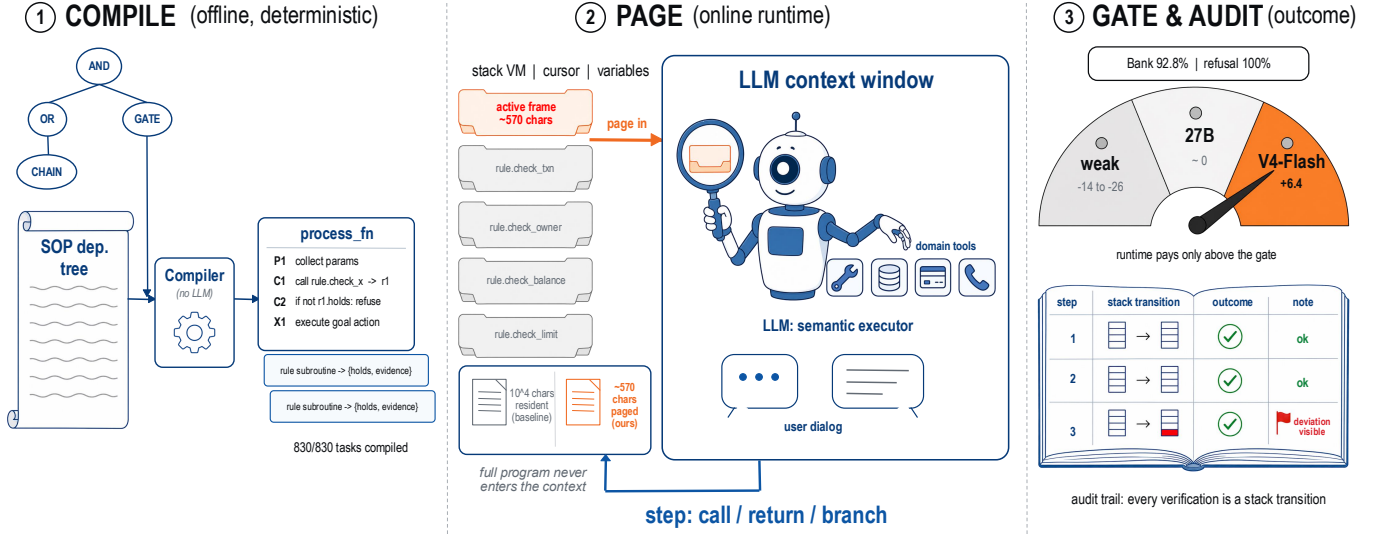


Figure 1: Overview. (1) An offline, LLM-free compiler turns the SOP dependency tree into a two-layer program (process functions calling rule subroutines that return $\{\text{holds}, \text{evidence}\}$; all 830 tasks compile). (2) A stack VM pages only the active frame (~ 570 vs. $\sim 10^4$ chars resident) while the LLM executes it semantically via one bookkeeping tool. (3) Benefits are capability-gated; every verification is an auditable stack transition; deviations execute (soft enforcement) but are flagged.

verdict assignment ν ,

$$\begin{aligned}
 \llbracket \ell \rrbracket_\nu &= \mathbf{1}\{\nu(c, \theta) = q\}, \\
 \llbracket \text{and}(\mathbf{T}) \rrbracket_\nu &= \llbracket \text{chain}(\mathbf{T}) \rrbracket_\nu = \bigwedge_i \llbracket T_i \rrbracket_\nu, \\
 \llbracket \text{or}(\mathbf{T}) \rrbracket_\nu &= \llbracket \text{gate}(\mathbf{T}) \rrbracket_\nu = \bigvee_i \llbracket T_i \rrbracket_\nu.
 \end{aligned} \tag{1}$$

Their truth conditions coincide in pairs, but their schedules do not: `chain` preserves source order; under our evidence-complete convention ordinary `or` evaluates every option before deciding; `gate` tests options in source order and stops at the first success.

Let $A_R(T)$ be the verifier actions referenced by T , $\Gamma(A)$ the login/authentication gates required by actions A , and $A_G(X)$ the actions that establish gates X . The hoisted guard set is the least fixed point

$$\begin{aligned}
 H_T &= \text{lfp}_X \Gamma(A_R(T) \cup A_G(X)), \\
 \hat{T} &= \text{chain}(\text{topo}(H_T \setminus L(T)), T).
 \end{aligned} \tag{2}$$

where $L(T)$ is the set of leaves already present in T ; duplicates are removed. Up to semantics-preserving inlining, reuse, and frame factoring, compilation is

$$\begin{aligned}
 \mathcal{C}(\ell) &= \text{Check}(\ell), \\
 \mathcal{C}(\text{and}(\mathbf{T})) &= \text{All}(\mathcal{C}(\mathbf{T})), \\
 \mathcal{C}(\text{chain}(\mathbf{T})) &= \text{All}^\rightarrow(\mathcal{C}(\mathbf{T})), \\
 \mathcal{C}(\text{or}(\mathbf{T})) &= \text{Any}^{\text{all}}(\mathcal{C}^*(\mathbf{T})), \\
 \mathcal{C}(\text{gate}(\mathbf{T})) &= \text{Any}^{\text{first}}(\mathcal{C}(\mathbf{T})).
 \end{aligned} \tag{3}$$

Here $\mathcal{C}^*$ is evidence-complete but truth-equivalent: Any^{all} disjoins after all options return, whereas $\text{Any}^{\text{first}}$ short-circuits. **Proposition 1** (Semantic and schedule preservation). *For a conforming execution whose rule returns are faithful to their*

cited tool results, $\mathcal{C}(\hat{T})$ returns $\llbracket \hat{T} \rrbracket_\nu$. Hence its prescribed control flow reaches the goal action iff the augmented SOP holds, and otherwise reaches fail first. Its trace preserves chain order, exhaustive ordinary-or evidence collection, and ordered gate short-circuiting, modulo same-frame reuse of an identical (rule, θ) result; it emits no predicate outside $L(T) \cup H_T$. The proof is a structural induction on Eq. (3); full cases and the fixed-point argument are in the supplement.

Technical advantages. All 830 artifacts load with closed call graphs and zero missing recipes. Compilation keeps the official leaf wording but adds recipes, guard closure, and evidence discipline; the content $\times$ format factorial isolates these additions. It also reduces the average actionable instruction surface from $\sim 10^4$ resident characters to a ~ 570 -character frame.

3.2 The Program-Guided Runtime

A monolithic compiled program still competes with the dialog and does not externally preserve position. The runtime therefore executes it as a stack machine driven through one tool, `pg_program_step`, with `start`, `call`, `return`, `select.branch`, `emit.artifact`, `complete`, and `fail`. It maintains a call stack, cursor, and returned-variable store, and discloses only the active frame into a task-local context slot. A `call` reveals the callee; a `return` restores the suspended caller with the result bound and its cursor advanced. State persists outside the dialog, so the frame can be re-disclosed after interruption or context compression. The LLM remains the semantic executor: it converses, chooses tool arguments, interprets evidence, and reports verdicts through `return`.

Process function `apply_credit_card` — *active frame* (573 chars)

```
P1: collect parameters (username,
total_assets, monthly_income); normalize to
the tool schema; ask if missing--never
fabricate.
C1: call rule.check_
username_exist(username) → result1
C2: if result1.holds ≠ true: refuse (cite
result1.evidence)
X1: once all checks hold, call
apply_credit_card(...) → outcome
X2: if outcome failed, relay the reason;
never retry with altered arguments
X3: report; return ({result: outcome})
```

Rule subroutine

```
rule.check_
username_exist

R1: constraint:
``user username must
exist``
R2: verify with
internal.get_
database() or
internal.check_
username_exist(
username)
R3: return ({holds,
evidence: {tool,
args, excerpt}})
No excerpt, no
verdict.
```

Stack audit trail
real transition log

```
start bank.apply_
credit_card [P1]
call rule.check_
username_exist [depth
2]
tool check → [true,
true]
return {holds:
true, evidence:
...} [X1]
tool apply_
credit_card(...) →
success [complete]
```

Figure 2: A real compiled artifact (Bank task `apply_credit_card#0`). The original’s Chinese structural scaffolding (Setup) is rendered line by line in English—structure, labels, identifiers verbatim. The runtime pages only the process frame (left) into context; its verification becomes an evidence-returning subroutine (middle) and an auditable stack transition (right).

Runtime semantics. A frame is $\phi = \langle f, \kappa, \eta, \rho \rangle$ (function, cursor, locals, caller return slot), and a configuration is $\sigma = (S, \Lambda)$ (frame stack and append-only trace). With the stack top on the right,

$$S \cdot \phi \xrightarrow{\text{call } g(x) \rightarrow z} S \cdot \phi \cdot \langle g, 0, x, z \rangle, \\ S \cdot \phi \cdot \langle g, \kappa', \eta', z \rangle \xrightarrow{\text{return } v} S \cdot \text{resume}(\phi, z, v). \quad (4)$$

Here $\text{resume}(\langle f, \kappa, \eta, \rho \rangle, z, v) = \langle f, \text{next}_f(\kappa), \eta[z \leftarrow v], \rho \rangle$, and each transition is appended to Λ . Thus nested calls preserve pending continuations; lexical paragraph retrieval alone is insufficient unless it also reconstructs equivalent continuation state. Define

$$R(\sigma) = \text{render}(\text{top}(S)), \quad B_P = \max_{f \in F(P)} |\text{render}(f)|. \quad (5)$$

The runtime injects only $R(\sigma)$, so $|R(\sigma)| \leq B_P$, independent of call depth and dialog length (Bank mean: ~ 570 vs. $\sim 10^4$ resident characters).

Proposition 2 (Well-bracketed auditability). *Every bound result has one dynamic producing **return**; all of its admissible evidence lies in the corresponding call–return interval. Consequently, a return without matching evidence, an out-of-cursor domain call, or a goal action before the accepting root cursor is detectable from the trace, although the runtime does not block it.* This follows immediately from the well-bracketed transitions in Eq. (4); one producer may serve multiple same-frame consumers through explicit reuse.

Technical advantages. Paging is call-graph-following, stateful, and revocable: a return removes the callee from scope and restores the caller with bound results. On Bank it reduces domain calls from 3.0 to 2.8 and raises refusal correctness to 100% for the strong workhorse (Results).

3.3 Soft Enforcement and Bounded Discretion

Hard blocking is brittle when goals change or evidence arrives in several valid orders. Our runtime therefore manages *attention*, not permissions: every action remains possible, while the stack makes skipped checks and unsupported returns observable. This flexibility predicts a capability threshold—models must faithfully execute the local frame—which Results tests directly. The compiler is deterministic and LLM-free (~ 0.1 s for 830 programs); the runtime adds one bookkeeping tool and persists state per task.

4 Experimental Setup

Benchmark and clean subset. We evaluate on SOPBench [Li et al., 2025], which casts SOP compliance as tool-use tasks over domain simulators with machine-readable dependency constraints and an oracle evaluator that checks both the executed call sequence (against a directed action graph) and the final database state. Bank is the primary attribution domain: 134 instances over 14 goals—*execute* tasks (preconditions satisfiable; act) and *refusal* tasks (unsatisfiable; refuse). Before comparing arms, an upstream zero-pass screen omitted 9 execute tasks from three Bank goals; current-source replay confirms simulator/state-transition defects for two, whereas *open_account* is an empirical exclusion rather than a proved impossibility (supplement). We report the resulting screened subset of 125 (39 execute, 86 refusal), and recompute official baselines on it. Conservative full-denominator scores treating all nine omissions as failures are `offtext/flat/pg`: 65.7/80.6/86.6 over 134; paired statistics are unchanged because the omissions are arm-independent. The runtime contrast additionally runs on the other six domains (Table 4) with per-domain budgets (deep domains: 1800 s, 5 turns). The same pre-run screen omitted one upstream-zero-pass execute task in each of Library

and DMV; current-source replay finds both reachable, so we do not call them confirmed defects.

Three arms: isolating representation from runtime. All arms share the agent loop, domain tools, simulator, and budgets, differing only in how the SOP reaches the model: **offtext** is the benchmark’s official verbalized constraint text, resident every turn (the SOP-as-prompt baseline); **flat** is the full text of our compiled program, resident every turn (compiled *representation* without the runtime); **PG** is the same compiled program executed by the PG runtime with just-in-time instruction paging (Method). The offtext→flat delta measures *compilation* and flat→PG the *runtime* effect. A diagnostic **full-cursor** arm uses the same cursor-guided runtime and places the active frame first while retaining the exact full compiled program below it as global reference; it bundles frame repetition/position, cursor guidance, and global visibility rather than isolating pure highlighting. Its V4 three-arm common set has $n=819$ across seven domains. All tests are paired (exact McNemar; exact binomial pooling; per-contrast, unadjusted), with no post-hoc adherence filtering.

Models. Six assistant models span capability tiers: DeepSeek-V4-Flash (low-cost workhorse) and Qwen3.7-Plus (strong), GPT-4o-mini and Qwen2.5-7B-Instruct (weak), plus a same-vendor probe pair (Qwen3.6-27B/-35B-A3B) added to resolve the transition band. The user simulator is DeepSeek-V4-Flash for all arms and models. Three diagnostic arms used only in the compilation analysis are defined in Results.

Agent loop and budgets. Every task runs with identical caps: 150 agent turns, an attention-window refresh every 50 actions, and fuses of 900 s wall clock, 30 domain calls, 3 user turns. Each run gets a fresh task identity; every record embeds a configuration fingerprint (all Bank results share one); outcomes are scored solely by the official evaluator.

Comparability disclosures. Relative to the official leaderboard, our simulator, user-turn budget (3 vs. ~ 10), and clean task set differ; the within-harness offtext arm therefore anchors every causal contrast. Sensitivity checks preserve the result structure: at the official turn budget, PG remains above flat (90.4 vs. 87.2); a tied-top official model is slightly *lower* in our offtext loop (79.2 vs. 82.4 official); and a cross-family simulator reproduces the three-arm ordering (72.8/83.2/89.6). Compiled arms use Chinese structural scaffolding with English leaves; English re-emissions reproduce flat at both endpoints, while the weak model’s isolated code-format harm attenuates from -12.8 to -5.6 (n.s.). Full protocols and discordants are in the supplement.

Model	offtext	flat	PG	flat:off (p)	PG:flat (p)
DS-V4-Flash	70.4	86.4	92.8	26:6 (<.001)	9:1 (.021)
Q3.7-Plus	85.6	85.6	88.8	9:9 (1.000)	6:2 (.289)
Q3.6-A3B	81.6	81.6	84.0	14:14 (1.000)	12:9 (.664)
Q3.6-27B	79.2	86.4	85.6	16:7 (.093)	4:5 (1.000)
Q2.5-7B	65.6	68.8	54.4	22:18 (.636)	14:32 (.011)
GPT-4o-mini	60.8	75.2	48.8	32:14 (.011)	9:42 (<.001)

Table 1: Three arms across six models (Bank clean subset, one harness, $n=125$ per arm; all cells pass %). Right columns: discordant pairs with exact McNemar p (compilation flat:offtext; runtime PG:flat); point effects are column differences. 27B/A3B: same-vendor transition probe pair; full names in Setup.

Official configuration	Pass %	Refusal %
o4-mini-high (FC)	82.4	90.7
GPT-4o (FC)	82.4	84.9
Qwen2.5-32B (ReAct)	82.4	90.7
Qwen2.5-7B (FC) [†]	5.2	—

Table 2: Top official SOPBench configurations, recomputed on the identical clean subset (the three tied-top configurations; the remaining fifteen range 70.4–80.8). [†]Official full-set leaderboard score (refusal split unavailable): the 7B cross-harness reference.

5 Results

5.1 Attribution: Compilation vs. Runtime

The first row of Table 1 decomposes the end-to-end gain on Bank (clean subset, DeepSeek-V4-Flash) into its two layers. Replacing the official SOP text with the *compiled program text* — same channel, no runtime — lifts pass rate from 70.4% to 86.4% (+16.0 points; 26:6, $p < 0.001$). Enabling the PG runtime on the same program adds +6.4 within our harness, reaching 92.8% on the clean subset (86.6% on the full 134-task set; 9:1, $p=.021$), with 100% refusal correctness (86/86; two-sided 95% lower bound 95.8%; reproduced in both re-runs), fewer domain calls (3.0→2.8), and wall clock 181/105/132 s. Our PG configuration exceeds every recomputed official configuration on both metrics (Table 2); we treat this as a qualified external reference, since simulator, turn budget, and loop differ (Setup). The execute/refusal decomposition: execution moves little (74.4→71.8→76.9) while refusal climbs 68.6→93.0→100—what compilation and paging chiefly buy a strong model is *correct refusal behavior*. The offtext column of Table 1 extends this across six models: flat is never significantly below offtext, and the compilation gain lands where official prose underperforms (+16.0/+14.4 significant; +7.2/+3.2 n.s.), vanishing where official prose already sits at flat level (85.6, 81.6).

5.2 The Runtime Gain is Capability-Gated

Across six models (Table 1), the runtime effect Δ_{rt} is not noise around zero: it spans a signed range from +6.4 (DeepSeek-V4-

Model	offtext	off-code	c-prose	Δ_{fmt}	Δ_{cont}
DS-V4-Flash	70.4	84.8	88.0	26:8 (.003)	27:5 (<.001)
Q2.5-7B	65.6	52.8	68.0	10:26 (.011)	21:18 (.749)

Table 3: Compilation 2×2: {official, compiled} content × {prose, code} format, offtext as reference. *off-code* re-syntaxes official text with byte-identical leaves (machine-checked); *c-prose* renders compiled semantics as prose. $\Delta_{\text{fmt}}/\Delta_{\text{cont}}$ vs. offtext; both carry the scaffold-language change (Setup); flat column in Table 1.

Domain	V4	Plus	27B	7B
Bank	9:1	6:2	4:5	14:32
Hotel	13:7	26:16	—	—
Library	2:1	3:4	9:6	5:19
DMV	4:1	5:3	0:4	10:43
Healthcare	16:6	20:3	18:3	14:54
Market	14:2	15:3	9:11	15:66
University	0:1	0:0	1:1	1:23
Pooled W:L	58:19	75:31	41:30	59:237
Pooled p	<.001	<.001	.235	<.001

Table 4: Runtime contrast across seven domains: PG:flat discordant pairs (wins:losses for PG). The two strong models are evaluated across all domains; transition/weak cells follow the screened sets in Setup. Pooled rows use exact McNemar tests.

Flash, $p=0.021$) and +3.2 (n.s.), through a same-vendor probe pair on opposite sides of zero (+2.4/−0.8, both n.s.), down to −14.4 ($p=0.011$) and −26.4 (GPT-4o-mini, $p<0.001$); we claim no independent capability ordering. Pooling the two strong models yields 15:3 discordants; a task-level permutation test preserving the cross-model correlation (the models share tasks) gives $p=0.016$; the two weak models are independently significantly negative. Benjamini–Hochberg across the twelve per-model contrasts retains both compilation gains and both weak-model harms but not the single-model +6.4—runtime claims rest on the pooled discordants and replicate signs. Because instances within a goal share a program template, we also re-test with a goal-clustered sign-flip permutation (all discordants of a goal flip together): every cross-domain claim survives clustering (V4 58:19 and Plus 75:31, both $p=.004$; weak 59:237 at $p\leq 10^{-4}$; GPT-4o-mini’s Bank harm at $p=.028$), while the Bank-only strong-pair 15:3 spans just seven goal clusters and is underpowered there ($p=.19$)—a further reason runtime claims rest on cross-domain pools. Goal aggregates are in the supplement and complete per-goal results in the artifact. Identical-configuration re-runs keep every model on the same side of zero: the strong gain recurs, transition effects remain near zero, and the severe weak-model harm recurs at −24.0 to −33.6 (all $p<0.001$); full runs are in the supplement. A decoupling arm then attributes the effect *within* the runtime: *flat+protocol* keeps the full program resident while mandating the same bookkeeping protocol—paging off. Its overhead is insignificant at all three tiers (V4-Flash 3:4; GPT-4o-mini

Split	n	flat	full-cursor	PG
All	819	88.2	91.0	92.9
Execute	277	91.7	90.3	91.0
Refusal	542	86.3	91.3	93.9

Table 5: Full-program cursor ablation on the V4 three-arm common set (pass %). Full-cursor keeps the active frame first and the complete compiled program below it. Overall discordants: full-cursor:flat 57:34 ($p=.021$); PG:full-cursor 36:20 ($p=.044$).

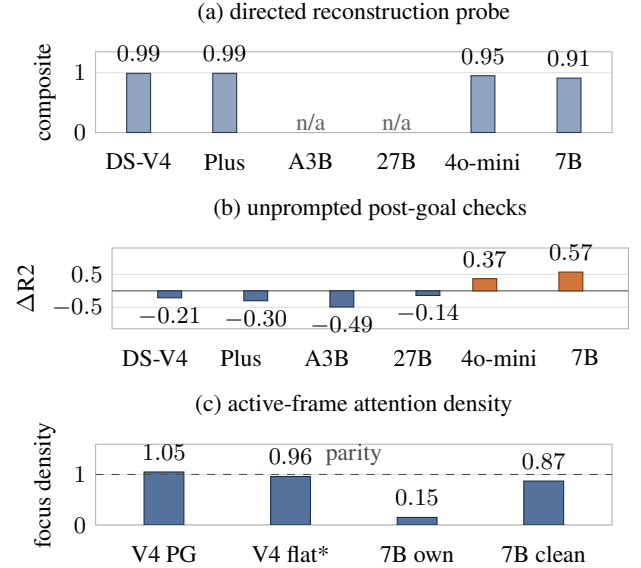


Figure 3: Mechanism. (a) Directed probes: near ceiling for all four models measured (transition pair not probed). (b) Wild audit ($\Delta R2$ = excess post-goal calls, PG—flat): the two harmed models are the two positive bars; the four unharmed are negative. (c) Attention over length share (1.0=parity): V4 at parity on its 2.1%-of-context frame; the 7B under-allocates to it on its own trajectories yet nears parity re-presented cleanly. *flat’s region is the whole program.

11:20; 7B 22:15; all n.s.), ruling out generic bookkeeping as the primary cause. Table 5 then retains the complete program but places the active frame first. On the V4 common set, flat/full-cursor/PG score 88.2/91.0/92.9: full-cursor beats flat 57:34 ($p=.021$), while PG further beats full-cursor 36:20 ($p=.044$). The separation is refusal-centered (86.3/91.3/93.9); execution is unchanged. Thus active-frame salience recovers much of the strong-model gain without hiding global instructions, and selective visibility adds a smaller increment.

The weak-model boundary is different: on Bank, 7B flat/full-cursor/PG score 68.8/55.2/54.4, and full-cursor is indistinguishable from PG (21:20, $p=1$). Restoring the full program therefore does not recover its loss, suggesting that active-frame-guided execution itself can exceed the model’s usable control capacity. GPT-4o-mini instead *over-verifies*, continuing checks after the goal action. Quantifying: 19 of its 64 PG failures (10 of 7B’s 57) are *outcome-correct* (right final state, no violation), failing only

the call-sequence check—absent under flat for every model; crediting all such cases still leaves it 11.2 below flat (64.0%): the gate is not a grading artifact.

5.3 Seven Domains across Capability Tiers

Table 4 extends the runtime contrast to all seven domains for two strong models, with transition and weak models on the five new domains. V4 pools to 58:19 ($p \approx 10^{-5}$), with six domains leaning PG; the independent Plus replication pools to 75:31 over 821 pairs ($p = 2.3 \times 10^{-5}$), with five domains leaning PG, Library 3:4, and University tied. This directly removes the single-strong-model dependence of the cross-domain finding without claiming a universal numerical threshold. The transition model pools to 41:30 (n.s.), splitting by domain; the weak model is harmed throughout, pooling to 59:237 ($p \approx 10^{-26}$; disjoint tasks, exact pools). No cell reverses its tier’s sign significantly. Healthcare rewards PG for both strong endpoints and the transition model (V4 16:6; Plus 20:3; 27B 18:3)—long but strictly tool-checkable chains. Deep-program domains (Hotel, up to 14 functions; University) show at most a modest strong-tier edge: on Hotel, PG passes 92.3% vs. flat 89.2% (13:7, n.s.) with the gain concentrated on refusal correctness (96.0% vs. 87.3%) while flat leads execution by 7.3 points—deep-domain constraints *look* inferable, inviting substitute-verification—at a wall-clock price (418 vs. 166 s/task).

5.4 Why Compilation Works: Content vs. Format

Compilation both *adds content* (verification recipes, verifier-gate closure, evidence discipline) and *re-formats* the policy as code. Table 3 separates the two with a 2×2 over {official, compiled} content $\times$ {prose, code}, anchored to the shared offtext baseline, using two equivalence-preserving transforms (*off-code*: official text re-syntaxed with byte-identical leaves; *c-prose*: compiled semantics as prose, same checks, order, gates, recipes; shared scaffolding conventions); the language ablations of Experimental Setup detect no significant language effect, and the off-code cell is measured directly: in English the weak model’s format harm attenuates to -5.6 (n.s.), direction unchanged.

Both factors are capability-dependent, in different ways: format flips sign, content pays only above a capability line. The *format* effect at fixed content flips sign across the capability range: re-syntaxing the official prose into code gains the strong model +14.4 points (26:8, $p = 0.003$) yet costs the weak model -12.8 (10:26, $p = 0.011$). The *content* effect at fixed prose format is +17.6 for the strong model (27:5, $p < 0.001$) but not detected for the weak one (+2.4, 21:18, $p = .75$); once content is compiled, format no longer matters (flat vs. c-prose: $-1.6/+0.8$, both n.s.). For the strong model, mechanical structure explicitation alone thus recovers most of compilation’s value; for the weak model, no representation manipulation significantly *improves* it within our harness (code re-syntaxing significantly harms it)—its dramatic lift over the official leaderboard (official full set 5.2%, $\approx 5.6\%$ rescaled, vs. 65.6% at our

offtext arm) is attributable to the execution harness, not the representation. Practically: the compiled *representation* never significantly hurt; isolated *content* pays above a low capability line; code *syntax* should be reserved for models it demonstrably helps.

A natural objection—the content effect merely leaks verification recipes—fails an ablation: appending complete recipes to the *unmodified* official text recovers a minority (70.4 $\rightarrow$ 76.8, n.s.), while compiled arms stay significantly ahead (flat +9.6, $p = .029$; c-prose +11.2, $p = .016$). Compilation buys not the recipe *information* but its *organization*: obligations, recipes, and evidence discipline in one goal-scoped executable flow.

5.5 Why the Runtime is Gated: Discipline, not Reconstruction

We probed the threshold with paired behavioral and white-box measurements (Figure 3).

Mechanism: capability present, deployment divergent. A directed state-reconstruction probe (machine-graded against the program cursor) scores *all* four probed models near ceiling (0.91–0.99, localization 100%), including both runtime losers: the gate is not inability to reconstruct state when asked.

An unprompted audit — R2, mean domain calls issued *after* the goal action already succeeded (successful-goal trajectories; no probe asked) — separates harmed from unharmed in the initial runs with a single sign rule, no exemptions: the two significantly harmed models are exactly the two with positive signatures (+0.37, +0.57); all four unharmed carry negative (-0.14 to -0.49). Both transition-band models, audited *after* the rule froze, landed unharmed—a prospective check on fresh trajectories. Across re-runs the rule separates *harm* robustly but its null region is noisy: the harmed signature recurs in all three runs (GPT-4o-mini: +0.37/ +0.12/ +0.48), while the strong models’ near-zero signatures drift positive in the second re-run (+0.09, +0.15)—a one-sided harm detector, not a two-sided predictor. Reconstruction capability is present wherever probed; *spontaneous* state discipline is what the runtime gate tracks.

Running the two open-weight endpoints on their own passing trajectories (architecture-faithful attention capture; real-token positions only): DeepSeek-V4-Flash spontaneously grants its 2.1%-of-context frame an at-parity share (1.05 vs. flat’s 0.96), and ablating the entire dialog history *raises* the gold action’s log-probability by 1.9 nats: the frame alone is sufficient to identify the target action. Qwen2.5-7B, in contrast, gives the frame only $0.15 \times$ parity on its own trajectories yet 0.87 re-presented cleanly: the capability–deployment gap at the attention level.

6 Discussion

Deployment guidance. The results reject “more scaffolding is better.” Compile when official prose underperforms; use code syntax only after a model-level check and paging only after

a discipline check. Strong models benefit from active-frame salience, with a smaller refusal-centered gain from selective visibility. For weak models, use the strong execution harness without active-frame guidance. Paging is soft and auditable, so irreversible actions still require hard tool-layer guards.

Limitations. (i) One benchmark (seven domains). (ii) Simulator/harness differences may leave residual affinity despite offtext, anchor, and turn-budget controls. (iii) Six models and no quantitative capability scale. (iv) Pass@1 without sampling-variance estimates; re-runs preserve each model’s gate side. (v) Machine-readable trees only; natural-language input unevaluated. An authorized, de-identified industrial 13-function/87-step case completed an employee-run workflow—qualitative portability evidence only, not compiler validation or matched comparison. (vi) Decoupling separates protocol from paging, but not state externalization from disclosure. Artifacts will be released.

7 Conclusion

We presented a deterministic SOP compiler and stack-paged runtime separating representation from visibility. Across six models and seven domains, two strong models independently benefit while weak models can be harmed. Full-cursor results recover most of the strong-model refusal gain through active-frame salience; selective visibility adds a smaller increment. Compile first; page only after a model-level discipline check.

References

- Anthropic. Equipping agents for the real world with agent skills. Engineering blog, October 16, 2025. URL <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>.
- Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for large language models. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 1946–1969, 2023. doi: 10.1145/3591300.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=sY5N0zY50d>.
- Zekun Li, Shinda Huang, Jiangtian Wang, Nathan Zhang, Antonis Antoniadis, Wenyue Hua, Kaijie Zhu, Sirui Zeng, Chi Wang, William Yang Wang, and Xifeng Yan. SOPBench: Evaluating language agents at following standard operating procedures and constraints, 2025. arXiv:2503.08669.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12: 157–173, 2024a. doi: 10.1162/tacl_a.00638.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024b.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023. arXiv:2310.08560.
- Yuchen Shi, Siqi Cai, Zihan Xu, Yuei Qin, Gang Li, Hang Shao, Jiawei Chen, Deqing Yang, Ke Li, and Xing Sun. FlowAgent: Achieving compliance and flexibility for workflow agents, 2025. arXiv:2502.14345.
- Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models, 2023. arXiv:2307.09702.
- Yiran Wu, Tianwei Yue, Shaokun Zhang, Chi Wang, and Qingyun Wu. StateFlow: Enhancing LLM task-solving through state-driven workflows. In *Conference on Language Modeling (COLM)*, 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=roNSXZpUDN>.
- Anbang Ye, Qianran Ma, Jia Chen, Muqi Li, Tong Li, Fujiao Liu, Siqi Mai, Meichen Lu, Haitao Bao, and Yang You. SOP-Agent: Empower general purpose AI agent with domain-specific SOPs, 2025. arXiv:2501.09316.
- Steve Young, Milica Gašić, Blaise Thomson, and Jason D. Williams. POMDP-based statistical spoken dialog systems: A review. *Proceedings of the IEEE*, 101(5):1160–1179, 2013. doi: 10.1109/JPROC.2012.2225812.
- Chenglin Yu, Yuchen Wang, Songmiao Wang, Hongxia Yang, and Li Ming. InfAgent: An infinite-horizon framework for general-purpose autonomous agents. In *Findings of the Association for Computational Linguistics: ACL 2026*,

pages 35884–35894, 2026. doi: 10.18653/v1/2026.findings-acl.1787.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.